

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4
ПО ДИСЦИПЛИНЕ «ТИПЫ И СТРУКТУРЫ ДАННЫХ»

Работа со стеком

Вариант № 3

Студент: **Чернышева А.П.**

Группа: **ИУ7-36Б**

Преподаватель: **Силантьева А. В.**

2025 г.

Описание условия задачи

Создать программу работы со стеком, выполняющую операции добавления, удаления элементов и вывод текущего состояния стека.
Реализовать стек:
а) статическим массивом;
б) списком.
Элементами стека являются слова.
Реализовать печать слов в обратном порядке, каждое слово в перевернутом виде.

Описание ТЗ

Описание исходных данных

Введенный пункт меню.
Ограничение — целое число от 0 до 9.
Слова для добавления в стек.
Ограничения:
- Максимальный размер стека массива: MAX_STACK_SIZE = 1000 элементов;
- Максимальная длина слова: MAX_WORD_LENGTH = 50 символов;
- Слова могут вводиться по одному или несколько через пробел.

Результаты:

Текущее состояние стека.
Слова в обратном порядке, каждое слово перевернуто.
Анализ эффективности реализации.
Освобожденные адреса памяти (для реализации списком).

Описание задачи, реализуемой программой

Программа реализует систему работы со стеком с возможностью:
1. Инициализация стека массивом
2. Инициализация стека списком
3. Добавление элемента в стек
4. Удаление элемента из стека
5. Просмотр текущего состояния стека
6. Печать слов в обратном порядке, каждое слово в перевернутом виде
7. Просмотр освобожденных адресов (только для списка)
8. Анализ эффективности
9. Демонстрация фрагментации памяти

Способ обращения к программе

Программа запускается через исполняемый файл app.exe.

Описание возможных аварийных ситуаций и ошибок пользователя

1. Переполнение стека
2. Удаление из пустого стека
3. Ввод пустой строки или строки состоящей только из пробельных символов
4. Неверный выбор пункта меню
5. Ошибка выделения памяти

Описание внутренних структур данных

ArrayStack - реализация стека массивом:

```
typedef struct {
    char data[MAX_STACK_SIZE][MAX_WORD_LENGTH];
    int top;
} ArrayStack;
```

data - статический массив для хранения элементов стека
top - индекс вершины стека (-1 если стек пуст)
ListNode – для хранения узла списка

```
typedef struct ListNode {
    char word[MAX_WORD_LENGTH];
    struct ListNode* next;
} ListNode;
```

word[MAX_WORD_LENGTH] слово
next указатель на предыдущий узел списка
ListStack реализация стека списком

```
typedef struct {
    ListNode* top;
    int size;
    void* freedAddresses[MAX_STACK_SIZE];
    int freedCount;
} ListStack;
```

top - указатель на вершину стека
size - текущий размер стека
freedAddresses - массив для отслеживания освобожденных адресов
freedCount - счетчик освобожденных адресов

Общий алгоритм программы:

Общий алгоритм программы

1. **Инициализация программы:**
   - Объявление структур для хранения стека в двух реализациях
   - Инициализация переменных для управления состоянием программы
2. **Основной цикл программы:**
   - Вывод меню с доступными операциями
   - Ожидание ввода выбора пользователя
   - Выполнение выбранной операции
3. **Операции меню:**

1 - Инициализация массива
2 - Инициализация списка

3 - Добавление элемента в стек
4 - Удаление элемента из стека
5 - Просмотр текущего состояния стека
6 - Печать слов в обратном порядке, каждое слово в перевернутом виде
7 - Просмотр освобожденных адресов (только для списка)
8 - Анализ эффективности
9 - Демонстрация фрагментации памяти
0 - Выход из программы

Описание алгоритмов функций

void initArrayStack(ArrayStack *s) - инициализация стека:

1. Установка указателя вершины стека в значение -1 (пустой стек)

int isArrayStackEmpty(ArrayStack *s) - проверка на пустоту:

1. Проверка равенства указателя вершины значению -1
2. Возврат 1 если стек пуст, 0 если содержит элементы

int isArrayStackFull(ArrayStack *s) - проверка на переполнение:

1. Проверка достижения указателем вершины максимального размера
2. Возврат 1 если стек полон, 0 если есть свободное место

int pushArrayStack(ArrayStack *s, char *word) - добавление элемента:

1. Проверка переполнения стека
2. Увеличение указателя вершины на 1
3. Копирование слова в массив по новому индексу
4. Возврат 0 при успехе, 1 при ошибке переполнения

void popArrayStack(ArrayStack *s, char *word) - удаление элемента:

1. Проверка пустоты стека
2. Копирование слова из вершины стека в выходной параметр
3. Уменьшение указателя вершины на 1

void printArrayStack(ArrayStack *s) - вывод стека:

1. Проверка пустоты стека
2. Последовательный вывод элементов от вершины к основанию
3. Отображение индекса и значения каждого элемента

void initListStack(ListStack *s) - инициализация стека:

1. Установка указателя вершины в NULL
2. Инициализация счетчика размера нулем
3. Очистка массива освобожденных адресов
4. Сброс счетчика освобожденных адресов

int pushListStack(ListStack *s, char *word) - добавление элемента:

1. Выделение памяти для нового узла
2. Проверка успешности выделения памяти
3. Копирование слова в новый узел
4. Установка связи нового узла с предыдущей вершиной
5. Обновление указателя вершины на новый узел
6. Увеличение счетчика размера стека

7. Возврат 0 при успехе, 1 при ошибке выделения памяти

void popListStack(ListStack *s, char *word) - удаление элемента:

1. Проверка пустоты стека
2. Копирование слова из вершины в выходной параметр
3. Сохранение адреса удаляемого узла в массив освобожденных адресов
4. Обновление указателя вершины на следующий элемент
5. Освобождение памяти удаляемого узла
6. Уменьшение счетчика размера стека

void printListStack(ListStack *s) - вывод стека:

1. Проверка пустоты стека
2. Последовательный обход всех узлов от вершины к основанию
3. Вывод адреса каждого узла и хранимого слова

void printFreedAddresses(ListStack *s) - вывод освобожденных адресов:

1. Проверка наличия освобожденных адресов
2. Последовательный вывод всех адресов из массива freedAddresses
3. Отображение порядкового номера и значения адреса

void reverseWord(char *word) - вывод слова в обратном порядке:

1. Определение длины слова
2. Последовательный вывод символов от конца к началу
3. Добавление пробела после слова

void processStack(ArrayStack *arrStack, ListStack *listStack, int useList) - обработка стека:

1. Проверка типа используемого стека (массив/список)
2. Последовательное извлечение всех элементов из стека
3. Переворот каждого извлеченного слова
4. Вывод перевернутых слов в порядке извлечения

Тесты

Позитивные тесты

Номер теста	Описание	Ожидаемый результат
1	Добавление слов "raise hand" в стек-массив	Стек содержит ["hand","raise"]
2	Добавление слов "make call save" в стек-список	Стек содержит ["save", "call","make"]
3	Печать слов в обратном порядке (массив) Стек: ["abc", "def"]	"fed cba"
4	Удаление элементов из заполненного стека ["one", "two", "three"]	Удален: three
5	Вывод массива	Успешный вывод массива.
6	Анализ эффективности	Таблица сравнения

		времени и памяти
7	Вывод списка освобожденных адресов	Успешный вывод списка освобожденных адресов
8	Добавление нескольких слов через пробел в стек списком Слова: "test stack program"	Стек содержит ["test", "stack", "program"]

Негативные тесты

Номер теста	Описание	Ожидаемый результат
1	Добавление в переполненный стек	"Стек переполнен"
2	Удаление из пустого стека	"Стек пуст"
3	Ввод пустой строки	" Строка пустая или состоит только из пробельных символов "
5	Просмотр освобожденных адресов для массива	"Только для стека-списка"
6	Просмотр освобожденных адресов для массива, если еще ничего не удалили	"Нет освобожденных адресов"
7	Ввод строки состоящей только из пробельных символов	"Строка пустая или состоит только из пробельных символов "
8	Некорректный выбор в меню.	Неверный выбор

Оценка эффективности

Замеры сделаны за 10000 операций. Время показано за одну операцию добавления-удаления.

Элементов	Время массива (нс/оп)	Время списка (нс/оп)	Массив быстрее на
10	14.97	22.50	33.5%
50	7.35	16.63	55.8%
100	6.86	16.39	58.1%
1000	6.85	17.89	61.7%

Элементов	Массив (байт)	Список (байт)	Память массива больше на
10	50004	8664	82.7%
50	50004	11224	77.6%
100	50004	14424	71.2%

1000	50004	72024	-44.0%

Реализация списком значительно эффективнее при частичном заполнении стека.

Вывод

Исследованы две структуры данных стека: на основе статического массива и односвязного линейного списка. Стек на массиве оказывается более эффективным по времени особенно для длинных последовательно стей. Его фиксированный размер и доступ по индексу обеспечивают высокую скорость операций, что позволяет обрабатывать данные быстрее, чем в стеке на списке, где работа с указателями создает дополнительную нагрузку. Массив в среднем на 52.3% быстрее списка. Реализация с писком значительно эффективнее по мамяти при частичном заполнении стека. При малых размерах (10-100 элементов) список экономит 77.2% памяти в среднем. При больших размерах (1000 элементов) массив становится на 44.0% эффективнее по памяти.

Контрольные вопросы

1. Что такое стек?

Стек - это последовательный список с переменной длиной, в котором включение и исключение элементов происходит только с одной стороны - с его вершины. Стек функционирует по принципу LIFO (Last In - First Out).

2. Каким образом и сколько памяти выделяется под хранение стека при различной его реализации?

При реализации массивом - выделяется непрерывная область памяти ограниченного размера. При реализации списком - память выделяется динамически для каждого элемента, объем ограничен только доступной оперативной памятью.

3. Каким образом освобождается память при удалении элемента стека при различной реализации стека?

В массиве - при удалении элемента указатель смещается, но память не освобождается. В списке - память явно освобождается после удаления элемента

4. Что происходит с элементами стека при его просмотре?

При классической реализации - просмотреть содержимое стека без извлечения элементов невозможно. В данной работе реализован специальный механизм для просмотра.

5. Каким образом эффективнее реализовывать стек? От чего это зависит?

Эффективность зависит от конкретных условий: массив эффективнее при известном размере и требовании к скорости, список - при неизвестном размере и ограниченной памяти.